

Integrating Kolmogorov-Arnold Networks into DreamerV3: A Principled Approach via Binary Latent Representations

Przemysław Sekula^{1,2} Przemysław Głomb¹ Michał Romaszewski¹
Anna Zawadzka¹ Michał Cholewa¹
Katarzyna Kołodziej¹

¹Institute of Theoretical and Applied Informatics, Polish Academy of Sciences, Gliwice, Poland

²University of Maryland, College Park, MD, USA

Abstract

World model-based reinforcement learning has achieved strong sample efficiency through the use of compact latent representations and model-based planning. DreamerV3 [7] is a leading example, using a Recurrent State-Space Model (RSSM) with discrete categorical latent variables. We investigate replacing the standard multilayer perceptron (MLP) networks in DreamerV3’s actor-critic module with Kolmogorov-Arnold Networks (KANs), which offer potential benefits for interpretability and, tentatively, for resilience to catastrophic forgetting. Direct substitution of MLPs with KANs inside DreamerV3’s actor-critic faces two distinct obstacles, however. The first is practical: KAN spline evaluation scales multiplicatively with input dimension, the actor-critic receives 1536-dimensional latent features, and it is evaluated at every step of every imagined rollout, making the computational overhead severe. The second is a knowledge gap: prior work had not identified a principled, Dreamer-specific way to alleviate this cost. We address both by identifying an exact structural property of Dreamer’s latent representations. The stochastic RSSM state is a binary one-hot vector, and for any such binary input the first KAN layer is mathematically equivalent to a standard affine transformation—spline expressivity is simply not available at two evaluation points. This motivates *Sparse-KAN*, a hybrid architecture that replaces the first KAN layer with a dense linear layer, exploiting the equivalence exactly, while applying full KAN layers to subsequent continuous representations where splines can express nontrivial functions. We validate this approach on the Atari 100k benchmark and demonstrate that the hybrid architecture successfully learns across a range of Atari games.

1 Introduction

Model-based reinforcement learning (MBRL) has emerged as a powerful paradigm for achieving sample-efficient learning in complex sequential decision-making tasks. By learning a world model that predicts how the environment evolves in response to actions, an agent can generate imagined experience in latent space and use it to improve its policy without additional interaction with the environment. DreamerV3 [7] is one of the most successful MBRL methods, demonstrating strong performance across domains as diverse as Atari games, continuous control, and 3-D environments using a single hyperparameter configuration. Its key architectural components are a Recurrent State-Space Model (RSSM) with discrete categorical latent variables, a convolutional encoder and decoder for visual observations, and a model-free actor-critic that operates entirely in the learned latent space.

Kolmogorov-Arnold Networks (KANs) were recently proposed by Liu et al. [12] as a principled alternative to multilayer perceptrons (MLPs). Rather than learning fixed nonlinearities

applied to linear projections of the input, KAN layers learn *univariate activation functions*—realized as learnable B-spline curves—separately for each connection in the network. This design has two compelling properties that motivate their use in reinforcement learning. First, the learned per-edge functions are directly interpretable as plots of input-output relationships, offering a path toward explainable neural decision-making. Second, because the spline functions are local (each B-spline basis function has compact support), updates to one part of the input space do not necessarily disturb representations elsewhere. This locality may improve the network’s resilience to catastrophic forgetting, a well-known failure mode in continual and non-stationary learning settings such as reinforcement learning, though empirical evidence for this advantage is mixed and appears task-dependent [19, 15].

Despite their promise, KANs face a significant practical obstacle: they are substantially more expensive to evaluate than equivalent MLPs. The spline computation introduces a multiplicative overhead proportional to the grid size and the number of input features, making direct substitution of MLP layers with KAN layers in high-dimensional networks computationally prohibitive. This challenge is especially acute in world model-based RL, where the actor and critic networks receive features of dimension $d_{\text{stoch}} + d_{\text{deter}} = 1024 + 512 = 1536$ in the default DreamerV3 configuration, and must be evaluated for every imagined transition over a planning horizon of 15 steps. Prior work integrating KANs into world-model-based RL has addressed this cost through implementation-level adaptations—vectorization, fixed grids, simplified initialization—but has not derived a principled justification rooted in the specific structure of Dreamer’s latent representations.

This paper closes both gaps simultaneously. Our key observation is that the RSSM stochastic state has a specific binary structure that renders the first KAN layer redundant. Concretely, the RSSM represents the stochastic component of the latent state as a collection of independent categorical variables: 32 groups, each represented as a one-hot vector of dimension 32, yielding a 1024-dimensional binary vector. The actor and critic networks receive this flattened one-hot representation as a dominant part of their input. We show analytically that, for inputs restricted to $\{0, 1\}^n$, a KAN linear layer is exactly equivalent to a standard affine transformation: the learnable spline functions collapse to linear functions of the binary input, and the net computation reduces to a weight matrix multiplication plus a bias term. This equivalence means that using a full KAN layer at the input of the actor-critic network wastes capacity—the expressive power of splines is not utilized, yet the spline computation is still performed.

Motivated by this observation, we design *Sparse-KAN*, a hybrid architecture in which the first layer of the actor and critic is replaced by a standard dense linear layer (exploiting the mathematical equivalence), and all subsequent layers are KAN layers (where inputs are no longer binary, so splines can express nontrivial functions). This design recovers the full expressive power of KANs in the intermediate representations without paying for their computational overhead at the binary input stage.

We evaluate Sparse-KAN on the Atari 100k benchmark [8], a standard testbed for sample-efficient visual reinforcement learning in which agents are limited to 4×10^5 environment steps. Our experiments show that the hybrid architecture successfully learns across a diverse set of Atari games, matching or approaching the performance of the standard MLP-based DreamerV3 baseline. To the best of our knowledge, this is the first work in Dreamer-style world model reinforcement learning to exploit the binary one-hot structure of the RSSM stochastic state and show that the first KAN layer collapses to an affine map, motivating a hybrid linear-plus-KAN actor-critic.

2 Related Work

World model-based reinforcement learning. World model-based reinforcement learning has developed into a strong paradigm for sample-efficient control by learning latent dynamics and

improving policies from imagined rollouts rather than only from direct environment interaction. Early work on *World Models* [3] showed that compact learned simulators can support downstream policy learning, while PlaNet [5] introduced recurrent state-space models with stochastic and deterministic latent components for planning in latent space. Dreamer [4] then demonstrated that actor and critic networks can be trained directly on imagined trajectories, avoiding explicit planning at decision time. This line culminated in DreamerV2 [6] and DreamerV3 [7], which established discrete latent world models as a highly effective basis for visual control and broad-domain reinforcement learning. In particular, DreamerV2 introduced discrete world models for Atari, and DreamerV3 showed that a single world-model agent can perform strongly across diverse domains with fixed hyperparameters.

Kolmogorov-Arnold Networks and their efficient variants. Kolmogorov-Arnold Networks (KANs) were proposed as an alternative to standard MLPs in which each input-output connection is parameterized by a learnable univariate function, typically implemented with spline-based bases, rather than by a single scalar weight. The original KAN work motivated this design through interpretability and local functional plasticity, and it triggered a rapidly growing literature on practical variants. Since the original formulation is computationally expensive, several follow-up works have focused on efficiency, including FastKAN [11], which approximates spline bases with radial basis functions, ReLU-KAN [14], which uses matrix-friendly ReLU constructions, UKAN [13], which addresses bounded-grid limitations, and Efficient-KAN [1], which provides a more memory-efficient implementation. At the same time, the empirical evidence on KANs remains mixed: recent careful comparisons suggest that KANs do not consistently outperform well-matched MLPs outside selected settings [19], and their advantages for catastrophic forgetting appear to be task-dependent rather than universal [15].

KANs in reinforcement learning. A growing body of work has already explored KANs as function approximators in reinforcement learning. Prior studies have evaluated KAN-based PPO in online control [9] and KAN-based actor-critic architectures in offline reinforcement learning [2], often finding that KANs can achieve performance comparable to MLP baselines while sometimes using fewer parameters. Other reinforcement-learning applications have emphasized interpretability, for example in network control and value-based learning settings. These studies show that the idea of using KANs in reinforcement learning is no longer novel by itself. Rather, the open question is whether KANs can be integrated effectively into *world model-based* reinforcement learning, where actor and critic networks are evaluated repeatedly during imagination rollouts and computational overhead becomes especially important.

Closest prior work. The closest prior work we found is *KAN-Dreamer* [17], which explicitly studies the integration of KAN and FastKAN components into DreamerV3. That work considers replacing MLP-based modules in visual perception, latent prediction, and behavior learning, and it reports that KAN-based actor-critic variants can train successfully in DreamerV3. Consequently, a broad claim that the present paper is the first successful integration of KANs into a world model-based RL agent would be too strong. However, the contribution of the present paper is different in emphasis and mechanism. Whereas prior Dreamer/KAN work motivates feasibility primarily through implementation-oriented choices such as vectorization, fixed grids, and simplified initialization, our approach is based on a structural property of Dreamer-style latent states: the stochastic RSSM component is represented as binary one-hot variables. This observation leads to a principled result showing that a KAN input layer acting on binary inputs reduces to an affine map, which in turn motivates replacing the first KAN layer with a standard dense transformation and reserving KAN layers for subsequent continuous representations. To the best of our knowledge, this specific binary-input equivalence and the resulting hybrid linear-plus-KAN design have not been previously identified in the Dreamer/KAN literature.

Positioning of the present work. Taken together, the literature suggests that the main novelty of the present paper is not the general use of KANs in reinforcement learning, nor the mere replacement of MLP components inside DreamerV3. Instead, the contribution lies in identifying a Dreamer-specific architectural simplification that follows from the binary one-hot structure of the discrete RSSM latent state, and in using that result to design a computationally feasible hybrid actor-critic for world model-based RL. This positions our work at the intersection of efficient world-model reinforcement learning and hybrid KAN architectures.

3 Methods

3.1 DreamerV3 Background

DreamerV3 learns a world model of the form:

$$\text{Encoder: } z_t \sim q_\phi(z_t \mid o_t, h_t) \quad (1)$$

$$\text{Recurrence: } h_t = f_\phi(h_{t-1}, z_{t-1}, a_{t-1}) \quad (2)$$

$$\text{Dynamics: } \hat{z}_t \sim p_\phi(\hat{z}_t \mid h_t) \quad (3)$$

$$\text{Decoder: } \hat{o}_t \sim p_\phi(\hat{o}_t \mid h_t, z_t) \quad (4)$$

$$\text{Reward / cont.: } \hat{r}_t, \hat{c}_t \sim p_\phi(\cdot \mid h_t, z_t) \quad (5)$$

where o_t is the observation, a_t is the action, z_t is the stochastic latent state, and h_t is the deterministic recurrent state. The recurrence f_ϕ is a Gated Recurrent Unit (GRU), making h_t the hidden state of the memory network.

Discrete latent variables. The stochastic state z_t is parameterized as a product of $N_s = 32$ independent categorical distributions, each over $N_d = 32$ categories. At each step, a single category is selected per group, yielding a one-hot encoding. The full stochastic state is the concatenation of these one-hot vectors, flattened to $z_t \in \{0, 1\}^{1024}$. Training uses the straight-through estimator to allow gradients to flow through the discrete sampling operation.

Feature representation. The complete state feature used by the actor and critic is the concatenation

$$\mathbf{f}_t = [z_t; h_t] \in \mathbb{R}^{1536}, \quad (6)$$

where $z_t \in \{0, 1\}^{1024}$ is the binary stochastic state and $h_t \in \mathbb{R}^{512}$ is the continuous GRU hidden state.

Behavior learning via imagination. DreamerV3 trains an actor π_θ and critic v_ψ entirely within the world model. Starting from an encoded state, the agent imagines a trajectory of $H = 15$ steps by repeatedly sampling actions from π_θ , advancing the dynamics with f_ϕ , and evaluating v_ψ to compute bootstrapped λ -returns. The actor is trained to maximize these imagined returns; the critic is trained to predict them. Both networks receive the feature $\mathbf{f}_t$ as input and are implemented as MLPs in the original DreamerV3.

3.2 Kolmogorov-Arnold Networks

Formulation. The Kolmogorov-Arnold representation theorem states that any multivariate continuous function can be expressed as a finite composition of univariate functions [10]. KANs operationalize this idea by replacing the learned scalar weights of a standard linear layer with

learned univariate activation functions, one per input-output connection. A single KAN linear layer with n inputs and m outputs computes:

$$y_k = \sum_{i=1}^n \phi_{ki}(x_i), \quad k = 1, \dots, m, \quad (7)$$

where each $\phi_{ki} : \mathbb{R} \rightarrow \mathbb{R}$ is a learnable function. In practice, each ϕ_{ki} is parameterized as the sum of a base activation and a learnable B-spline [12]:

$$\phi_{ki}(x) = w_{ki}^{\text{base}} \cdot b(x) + \sum_{j=1}^{G+p} c_{kij} \cdot B_j(x), \quad (8)$$

where $b(\cdot) = \text{SiLU}(\cdot)$ is the base activation, $\{B_j\}_{j=1}^{G+p}$ is a B-spline basis of order p defined over a uniform grid of $G+1$ knots on the interval $[-1, 1]$, and $\{c_{kij}\}$ are the learnable spline coefficients. The parameter w_{ki}^{base} controls the strength of the residual linear component.

Computational overhead. Evaluating (8) for a single layer requires computing $n \times m$ B-spline evaluations, each involving $G+p$ basis functions. For a high-dimensional input ($n = 1536$ in our setting) and typical grid size ($G = 8$) and spline order ($p = 3$), this introduces an order-of-magnitude increase in floating-point operations compared to an MLP layer of the same width. This overhead motivates the architectural optimization described next.

Efficient sparse KAN. We build on the efficient KAN implementation of [1], which introduces a sparse B-spline computation that skips evaluations for zero entries in the input tensor, exploiting the assumption $f(0) = 0$ for the spline component. This sparse formulation is particularly effective for binary inputs, where the majority of entries are zero.

3.3 Binary Input Equivalence

The binary nature of the discrete latent state z_t has an important consequence for any KAN layer that receives $\mathbf{f}_t$ as input.

Proposition 1 (KAN linear layer with binary inputs is affine). *Let $\mathbf{x} \in \{0, 1\}^n$ be a binary input vector. Then the output of a KANLinear layer as defined in (8) is an affine function of $\mathbf{x}$; that is, there exist $W^{\text{eff}} \in \mathbb{R}^{m \times n}$ and $\mathbf{b}^{\text{eff}} \in \mathbb{R}^m$ such that*

$$\text{KANLinear}(\mathbf{x}) = W^{\text{eff}} \mathbf{x} + \mathbf{b}^{\text{eff}}. \quad (9)$$

Proof. Fix any output unit k and input feature i . Since $x_i \in \{0, 1\}$ and the spline grid is fixed, each B-spline basis function B_j evaluates to one of two constants at the input: $B_j(0)$ or $B_j(1)$. Likewise, $b(0) = \text{SiLU}(0) = 0$ and $b(1) = \text{SiLU}(1)$ are constants independent of the learnable parameters. Using the identity $B_j(x_i) = x_i B_j(1) + (1 - x_i) B_j(0)$ and $b(x_i) = x_i b(1)$, we expand (8):

$$\begin{aligned} \phi_{ki}(x_i) &= w_{ki}^{\text{base}} x_i b(1) + \sum_j c_{kij} [x_i B_j(1) + (1 - x_i) B_j(0)] \\ &= x_i \underbrace{\left[w_{ki}^{\text{base}} b(1) + \sum_j c_{kij} (B_j(1) - B_j(0)) \right]}_{W_{ki}^{\text{eff}}} + \underbrace{\sum_j c_{kij} B_j(0)}_{\beta_{ki}}. \end{aligned} \quad (10)$$

Summing over all input features:

$$y_k = \sum_{i=1}^n \phi_{ki}(x_i) = \sum_{i=1}^n W_{ki}^{\text{eff}} x_i + \underbrace{\sum_{i=1}^n \beta_{ki}}_{b_k^{\text{eff}}}, \quad (11)$$

which is the k -th component of $W^{\text{eff}}\mathbf{x} + \mathbf{b}^{\text{eff}}$. $\square$

The proposition shows that applying a KAN layer to binary inputs is no more expressive than applying a standard linear layer: with only two evaluation points per function ($\{0, 1\}$), the learnable splines cannot represent any nonlinearity. Any function capacity stored in the spline coefficients is therefore redundant with the capacity of a linear weight matrix. Consequently, using a full KAN as the first layer of the actor or critic—when its input contains a dominant binary component from the discrete latent state—wastes parameters and computation without providing any representational benefit.

3.4 Sparse-KAN: Hybrid Architecture

Architecture. Based on Proposition 1, we design the Sparse-KAN module used for both the actor and the critic. Given the input feature $\mathbf{f}_t \in \mathbb{R}^{1536}$, the network computes:

$$\mathbf{h}_0 = \text{SiLU}(W_0 \mathbf{f}_t + \mathbf{b}_0), \quad \mathbf{h}_\ell = \text{KAN}_\ell(\mathbf{h}_{\ell-1}), \quad \ell = 1, \dots, L, \quad (12)$$

where $W_0 \in \mathbb{R}^{d_0 \times 1536}$ is a standard trainable weight matrix and KAN_ℓ are KAN linear layers as defined in (8). The first transformation is an ordinary dense linear projection followed by a SiLU nonlinearity; subsequent layers are full KAN layers that exploit the expressive power of learnable splines over continuous intermediate representations.

This design is mathematically justified by Proposition 1: replacing the first KAN layer with a dense linear layer is exact for the binary portion of the input. The continuous portion h_t enters the network jointly through the same first projection, and nonlinear processing by the spline functions begins in the second layer once the activations are no longer binary.

Layer widths and spline parameters. For Atari 100k experiments we use hidden widths $[d_0, d_1, d_2] = [128, 64, 32]$, B-spline grid size $G = 8$, and spline order $p = 3$ (cubic splines). The same architecture is used for both actor and critic. The output heads (action logits for the actor, value distribution logits for the critic) are standard linear layers applied to $\mathbf{h}_L$.

Action distribution. In Atari, actions are discrete; the actor samples from a one-hot categorical distribution using the REINFORCE gradient estimator [18]. This means the action vectors that enter the RSSM as previous actions a_{t-1} are also binary one-hot vectors, further motivating the linear first-layer treatment for inputs to the dynamics model.

3.5 Training Setup

Environment and benchmark. We evaluate on the Atari 100k benchmark [8], in which agents are trained for 4×10^5 environment steps (10^5 steps with action repeat 4) and evaluated over 100 episodes. We train on 26 Atari games using standard preprocessing: grayscale observations, no-op initialization with up to 30 no-ops, and episode termination on life loss disabled.

World model. The world model follows the DreamerV3 default for Atari 100k: a convolutional encoder and decoder with 32 channels, RSSM with $N_s = 32$ stochastic categories of dimension $N_d = 32$ (so $z_t \in \{0, 1\}^{1024}$) and deterministic hidden dimension $d_h = 512$, trained with a replay ratio of 1024 updates per environment step. The world model components (encoder, RSSM, decoder, reward head, continuation head) use standard MLP layers as in the original DreamerV3, unchanged.

Actor and critic. Only the actor and critic networks are replaced with the Sparse-KAN architecture described above. Isolating the replacement to the behavior module allows us to measure the effect of KAN on policy learning independently of any change in world model quality. The actor uses REINFORCE; the critic uses a symlog-encoded two-hot distribution over returns.

Optimization. All parameters are optimized with Adam. The world model uses learning rate 10^{-4} ; the actor and critic use 3×10^{-5} . Gradient norms are clipped at 10^3 for the world model and 10^2 for the behavior module. We use a batch size of 16 sequences of length 64.

Baseline. The baseline is the standard DreamerV3 with MLP actor and critic, identical in all hyperparameters except for the network architecture. Performance is reported as mean and standard deviation of undiscounted episode returns across seeds and evaluation episodes.

4 Results

4.1 Per-game Performance

Table 1 reports the human-normalized score (HNS) for each of the 26 Atari 100k games at the end of training for both the DreamerV3 baseline (MLP actor-critic) and the proposed Sparse-KAN actor-critic. All other components of the agent—the world model, encoder, decoder, and training hyperparameters—are identical between the two conditions.

Table 1: Human-normalized scores (HNS) at the final checkpoint across 26 Atari 100k games. Scores above 1 exceed the reference human level. Bold indicates the higher score for each game. [†]Breakout data is unavailable for Sparse-KAN; mean is computed over the remaining 25 games.

Game	DreamerV3 (baseline)	Sparse-KAN (ours)	Game	DreamerV3 (baseline)	Sparse-KAN (ours)
Alien	0.150	0.080	Kangaroo	1.355	0.586
Amidar	0.086	0.037	Krull	6.177	7.537
Assault	1.060	1.175	Kung Fu Master	1.138	1.022
Asterix	0.111	0.099	Ms. Pac-Man	0.279	0.057
Bank Heist	1.231	0.434	Pong	1.093	1.153
Battle Zone	0.215	0.407	Private Eye	− 0.003	−0.000
Boxing	6.905	7.352	Qbert	0.182	0.035
Breakout	0.938	N/A [†]	Road Runner	1.870	1.940
Chopper Cmd.	0.199	0.040	Seaquest	0.014	0.003
Crazy Climber	2.790	2.670	Up N Down	0.942	0.491
Demon Attack	0.048	0.062			
Freeway	0.216	0.000			
Frostbite	0.561	0.045			
Gopher	1.118	1.298			
Hero	0.169	0.079			
Jamesbond	1.542	1.507			
Mean HNS (25 games [†])				1.169	1.124
Median HNS				0.750	0.407
Games > human				11	9

4.2 Comparison with Published Methods

Table 2 places our two conditions alongside representative published methods on Atari 100k, drawn from [16]. Because computing the interquartile mean (IQM) robustly requires per-seed, per-game score distributions that we do not have for all baselines, we report mean and median HNS for all methods, using mean as the primary metric. Published values for the literature baselines are taken directly from Table A.1 of [16].

Table 2: Aggregate human-normalized scores on Atari 100k (26 games), sorted by mean HNS. Values for literature methods are taken from [16]; values for our two conditions are computed from the final training checkpoint averaged across seeds. Higher is better. [†]Sparse-KAN mean is over 25 games (Breakout data unavailable).

Method	Mean HNS	Median HNS
Random	0.000	0.000
DER	0.350	0.189
DrQ(ϵ)	0.465	0.313
SPR	0.616	0.396
IRIS	1.046	0.289
Sparse-KAN (ours)[†]	1.124	0.407
DreamerV3 (baseline, ours)	1.169	0.750
SR-SPR	1.272	0.685
EfficientZero	1.945	1.116
BBF	2.247	0.917
Human	1.000	1.000

4.3 Discussion

The method clearly works. Sparse-KAN successfully learns non-trivial policies across all 26 Atari 100k games, achieving super-human performance on 9 of them. A mean HNS of 1.12 places it just above IRIS and just below the DreamerV3 MLP baseline (Table 2), despite using a KAN architecture that was never tuned for performance. Some of this mean is driven by games where both conditions dramatically exceed human level (most notably Boxing and Krull, where spline-based representations appear particularly effective), but the breadth of learning—positive returns on every game—confirms that the KAN-based actor-critic is a viable function approximator in the DreamerV3 imagination rollout loop.

Comparison with the MLP baseline. Sparse-KAN underperforms the DreamerV3 MLP baseline (mean 1.12 vs. 1.17), a gap that is visible across most individual games in Table 1. This is not surprising. The MLP baseline uses the same width configuration that was tuned for DreamerV3 over many experiments by the original authors, while our KAN architecture—hidden widths [128, 64, 32] with $G = 8$ spline grid and cubic splines—was selected primarily to be computationally feasible rather than empirically optimal. Importantly, the Sparse-KAN architecture still matches or exceeds the baseline on several games (Kung Fu Master, Up N Down, Jamesbond), which shows that the KAN layers are capable of learning useful value and policy functions when the representation is continuous, exactly as the theory in Section 3.3 predicts.

Computational constraints and room for optimization. Due to hardware limitations, we were unable to conduct a systematic search over KAN architecture hyperparameters. The

hidden-layer widths, spline grid size, spline order, and number of KAN layers were all fixed without tuning. Prior comparative studies suggest that KAN performance is sensitive to these choices [19], and it is plausible that the current configuration is suboptimal. In particular, wider hidden layers or a finer spline grid (larger G) may allow the KAN layers to capture more expressive value and policy functions in the continuous intermediate representations where splines are beneficial. We therefore view the 1.12 mean HNS as a lower bound on what the Sparse-KAN architecture can achieve with proper hyperparameter tuning, and we expect that the gap to the MLP baseline is reducible—possibly substantially—with additional compute for architecture search.

References

- [1] Blealtan. Efficient-KAN. <https://github.com/Blealtan/efficient-kan>, 2024.
- [2] Haihong Guo, Fengxin Li, Jiao Li, and Hongyan Liu. KAN v.s. MLP for offline reinforcement learning. *arXiv preprint arXiv:2409.09653*, 2024.
- [3] David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018.
- [4] Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations*, 2020.
- [5] Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *International Conference on Machine Learning*, 2019.
- [6] Danijar Hafner, Timothy Lillicrap, Mohammad Norouzi, and Jimmy Ba. Mastering Atari with discrete world models. In *International Conference on Learning Representations*, 2021.
- [7] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *Transactions on Machine Learning Research*, 2024. arXiv:2301.04104.
- [8] Łukasz Kaiser, Mohammad Babaeizadeh, Piotr Miłoś, Błażej Osiniski, Roy H. Campbell, Konrad Czechowski, Dumitru Erhan, Chelsea Finn, Piotr Kozakowski, Sergey Levine, Afroz Mohiuddin, Ryan Sepassi, George Tucker, and Henryk Michalewski. Model-based reinforcement learning for Atari. In *International Conference on Learning Representations*, 2020.
- [9] Victor Augusto Kich, Jair Augusto Bottega, Raul Steinmetz, Ricardo Bedin Grando, Ayano Yoroza, and Akihisa Ohya. Kolmogorov–Arnold network for online reinforcement learning. *arXiv preprint arXiv:2408.04841*, 2024.
- [10] Andrei N. Kolmogorov. On the representation of continuous functions of many variables by superposition of continuous functions of one variable and addition. *Doklady Akademii Nauk SSSR*, 114(5):953–956, 1957.
- [11] Ziyao Li. Kolmogorov–Arnold networks are radial basis function networks. *arXiv preprint arXiv:2405.06721*, 2024.
- [12] Ziming Liu, Yixuan Wang, Sachin Vaidya, Fabian Ruehle, James Halverson, Marin Soljačić, Thomas Y. Hou, and Max Tegmark. KAN: Kolmogorov–Arnold networks. *arXiv preprint arXiv:2404.19756*, 2024.

- [13] Alireza Moradzadeh, Lukasz Wawrzyniak, Miles Macklin, and Saeed G. Paliwal. UKAN: Unbound Kolmogorov–Arnold network accompanied with accelerated library. *arXiv preprint arXiv:2408.11200*, 2024.
- [14] Qi Qiu, Tao Zhu, Helin Gong, Liming Chen, and Huansheng Ning. ReLU-KAN: New Kolmogorov–Arnold networks that only need matrix addition, dot multiplication, and ReLU. *arXiv preprint arXiv:2406.02075*, 2024.
- [15] Mohammad Marufur Rahman, Guanchu Wang, Kaixiong Zhou, Minghan Chen, and Fan Yang. Catastrophic forgetting in Kolmogorov–Arnold networks. *arXiv preprint arXiv:2511.12828*, 2025.
- [16] Max Schwarzer, Johan Obando-Ceron, Aaron Courville, Marc G. Bellemare, Rishabh Agarwal, and Pablo Samuel Castro. Bigger, better, faster: Human-level Atari with human-level efficiency. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 30365–30380. PMLR, 2023.
- [17] Chenwei Shi and Xueyu Luan. KAN-dreamer: Benchmarking Kolmogorov–Arnold networks as function approximators in world models. *arXiv preprint arXiv:2512.07437*, 2025.
- [18] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256, 1992.
- [19] Runpeng Yu, Weihao Yu, and Xinchao Wang. KAN or MLP: A fairer comparison. *arXiv preprint arXiv:2407.16674*, 2024.

A Per-Game Training Curves

The following figures show evaluation return over the course of training for all 26 Atari 100k games. Each plot displays the mean return (solid line) $\pm$ one standard deviation across seeds (shaded band). **Blue**: DreamerV3 with MLP actor-critic (baseline). **Red**: Sparse-KAN actor-critic (ours). The x-axis represents environment steps; each checkpoint corresponds to 10 000 steps for a total of 4×10^5 steps. Breakout data is unavailable for Sparse-KAN (see footnote in Table 1).





